

ENEE 467: Robotics Project Laboratory

Dynamic Programming and Tabular Reinforcement Learning

Lab 8

Learning Objectives

The purpose of this lab is to provide a first hands-on experience with reinforcement learning using the Gymnasium framework. After completing this lab, students should be able to:

1. Implement model-based RL algorithms: value iteration and policy iteration, to understand the RL loop, convergence, and exploration-exploitation tradeoffs.
2. Implement model-free RL algorithms: Q-Learning and SARSA, to understand temporal-differencing, tune hyperparameters and compare performances of various RL models.

Reinforcement learning (RL) studies how an agent selects actions to maximize long-term rewards. In the tabular setting, states and actions are explicitly enumerated, allowing us to compute value functions, update policies, and evaluate state-action returns directly. This lab introduces these core ideas through hands-on implementation of dynamic programming and classical tabular RL methods, forming the groundwork for later study of continuous-state and deep RL techniques.

Related Reading

- I. R. S. Sutton and A. G. Barto, "Reinforcement Learning: An Introduction" (Second Edition), MIT Press, 2018. [Open Access](#).
- II. Lecture 14, "Introduction to Reinforcement Learning," ENEE 467: Robotics Project Laboratory, University of Maryland, College Park, MD, Nov. 2025.

Lab Instructions

Follow the lab procedure closely. Some code blocks have been populated for you and some you will have to fill in yourself. This lab includes one standalone question; the remaining work appears as **Exercises** after the environment setup. Work through each exercise and answer the accompanying questions. Complete all exercises during the lab and include your solutions in your lab writeup.

List of Questions

Question 1.	4
--------------------	---

List of Exercises

Exercise 1. Tabular Model-Based RL (35 pts.)	5
Exercise 2. Tabular Model-Free RL (50 pts.)	6

1 Lab Procedure

1.1 Software Installation

To prepare your system for the labs, install the required software using the command-line interface. Your machine should be running Ubuntu 20.04 or later to ensure compatibility with all libraries and drivers. If not, consider a virtual machine (VirtualBox, VMware, WSL2) or use the UMIACS computing service (recommended). We assume teams use UMIACS, though the instructions work natively on any Ubuntu 20.04+ system.

1.1.1 Getting this Lab's Code

To begin, connect to a Nexus cluster using your UMD Directory ID (`<umd_id>`) using:

```
ssh -Y <umd_id>@nexusclass.umiacs.umd.edu
```

Then from your current remote session, create a `Labs` folder within your home directory, move to it, and clone the lab repository (ignore the askpass Gtk-WARNING):

```
mkdir ~/Labs && cd ~/Labs
git clone https://github.com/ENEE467-F2025/lab-8.git
```

1.1.2 RL Environment Setup

To prepare your remote machine for the RL labs, create a `conda` environment using [Miniconda](#). From a terminal, go to your home directory with `cd ~` and run:

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
```

Confirm each prompt, and allow the installation to complete. When it's finished, restart your current shell by running `exec $SHELL`. You should then see the base environment activated:

```
(base) bash-4.4
```

Deactivate it using the following command:

```
conda deactivate
```

Then create a new `conda` environment named `rl467` and activate it by running the following:

```
export CONDA_PLUGINS_AUTO_ACCEPT_TOS="yes"
conda create --name rl467 python=3.11 -y
conda activate rl467
```

Continue by installing the [Gymnasium](#) library for RL tasks and a few ancillary libraries:

```
conda install -c conda-forge swig -y
conda install -c conda-forge matplotlib -y
pip install gymnasium[box2d]
```

1.1.3 Creating and Editing Files on a Remote Server

To create and edit files on a remote machine, there are several workflows, but we outline two of the most relevant to our context. You are free to use whichever workflow you find most convenient.

Option A: Editing Files with the Vim Text Editor

On remote machines without GUI-based editors, terminal-based text editors provide an effective alternative. One such tool is [Vim](#). You may start it directly with `vim`, or create and open a new file:

```
vim a.txt # create and open a.txt
```

This opens an empty Vim buffer in the terminal. By default you cannot type, so press `Shift+ A` to enter INSERT mode. Copy and paste work as in the Bash shell (e.g., `Ctrl+Shift+c`, `Ctrl+Shift+v`). Vim offers many editing shortcuts (see the [Vim Cheat Sheet](#)), but we will skip them. After editing, exit with `Ctrl+C`, then type `:wq` and press Enter to save and close Vim (in contrast, entering `:qa` + Enter will close Vim without saving the file).

Option B: Starting Remote Sessions From VSCode

For a more familiar development workflow and to avail yourself the opportunity of using other IDE features while developing on the remote machine, starting a remote session directly from VSCode is a convenient choice. Begin by installing the [Remote-SSH](#) extension on your host machine by navigating to `View → Extensions`, searching for Remote-SSH, and clicking on Install.

After installing the extension, connect to the Nexus cluster by executing the steps outlined at the following Microsoft guide: [Connect to a Remote Host](#). While following the guide, ensure that you use the appropriate SSH connection string as outlined in [Section 1.1.1](#). Once connection with the remote machine is established, VS Code will display the label, `SSH: nexusclass.umiacs.umd.edu` in the lower-left corner, and you should then be able to open folders on the remote machine from within the IDE. Typing `Ctrl + `` within the connected VSCode session should open up a remote Bash shell within VSCode's integrated terminal.

1.2 Test Your Installation

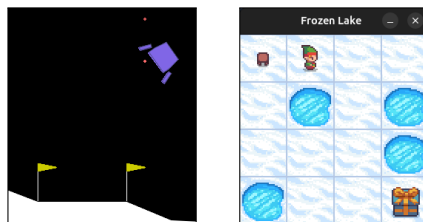


Figure 1: LunarLander and FrozenLake environments in Gymnasium.

Create a new Python file named `test_gym.py` in the `lab-8` repo directory:

```
cd ~/Labs/lab-8
touch test_gym.py # or vim test_gym.py, if you prefer
```

Copy-paste the following code template into Vim/the file within your Remote VSCode session:

```
import gymnasium as gym # main library providing sandboxes and RL utilities
import warnings
warnings.filterwarnings("ignore", category=UserWarning, module="pygame.pkgdata")

# Initialize the environment
env = gym.make("LunarLander-v3", render_mode="human")
print("Action space:", env.action_space)
print("Observation space:", env.observation_space)

# Reset the environment to generate the first observation
observation, info = env.reset(seed=42)
for _ in range(1000):
    # this is where you would insert your policy
    action = env.action_space.sample()

    # step (transition) through the environment with the action
    # receiving the next observation, reward
    # and if the episode has terminated or truncated
    observation, reward, terminated, truncated, info = env.step(action)

    # If the episode has ended then we can reset to start a new episode
    if terminated or truncated:
        observation, info = env.reset()
env.close()
```

Cross-check the file's indentation, save it, then run it using the following command:

```
python3 ~/Labs/lab-8/test_gym.py
```

You should see the text: "Action space: Discrete(4) ... Observation space: Box...."

TROUBLESHOOTING TIPS

a. MESA-LOADER Error:

If you encounter the integrated-graphics error `'libGL error: MESA-LOADER: failed to open iris'`, install:

```
conda install -c conda-forge libstdcxx-ng -y
```

b. Reactivating Your Conda Environment:

To reactivate the `rl467` environment in a new or reconnected session, run:

```
source ~/miniconda3/bin/activate && conda activate rl467
```

c. ALSA Driver Warnings:

These be safely ignored, as they do not affect execution of the programs we provide.

To familiarize yourself with the Gymnasium-style API, the code snippet in `test_gym.py` implements a random policy that uniformly samples actions in the LunarLander environment, but note that Gymnasium provides many environments to explore, and you are encouraged to experiment with additional environments available at [Gymnasium](#).

Question 1. Modify `test_gym.py` to reflect the `FrozenLake` environment by changing the environment ID within the `gym.make` call assigned to `env`. What are the action and observation space dimensions? (**Hint:** Print `env.action_space.n` and `env.observation_space.n`)

2 Exercises

Exercise 1. Tabular Model-Based RL (35 pts.)

In this exercise, you will implement tabular model-based RL algorithms—value iteration and policy iteration—for a grid-world setting using dynamic programming. The provided file `dynamic_programming.py` contains the skeletal code for the `FrozenLake` environment where you will implement the algorithms. The tasks to be completed are as follows:

1. Complete the `compute_action_values` function (marked with a `TODD`) so that it returns the correct action values, $Q(s, a)$, stored in the NumPy array `action_values`, using Bellman’s equation (see p. 17 of Ref. II in [Related Reading](#)). For reference, Bellman’s equation is:

$$V^\pi(s) = \sum_a \pi(a | s) \overbrace{\sum_{s', r} p(s', r | s, a) [r + \gamma V^\pi(s')]}^{Q(s, a)}, \quad (1)$$

where $\pi(a | s)$ gives the probability of selecting action a in state s . In the code, we treat a as an index drawn from the discrete action set of size `n_actions`, i.e. $a \in \{0, 1, \dots, \text{code{n_actions}-1}\}$. Thus, given the current state `state` (i.e., s), action index `a`, discount factor `gamma` (i.e., γ), and the value function V^π stored in the array `V` (of length `n_states`), the transition probability $p(s', r | s, a)$ (`prob`), next state s' (`next_state`), and reward r (`reward`) needed to compute `action_values` may be obtained via:

```
prob, next_state, reward, _ = env.unwrapped.P[state][a]
```

Finally, note that `compute_action_values` computes only $Q(s, a)$ (the blue term in (1)) for each action index; the policy $\pi(a | s)$ is applied later during policy evaluation. This keeps the function generic for both value iteration and policy iteration.

2. Complete the code for both value iteration (VI) and policy iteration (PI) by modifying the `TODD`-highlighted blocks in the methods `value_iteration` and `policy_iteration`.

Hints and Pseudocode. For VI, see Ref. II in [Related Reading](#) (pg. 20) and the Value Iteration page in Ref. I: [Value Iteration](#). There, `theta` corresponds to the lecture-slides tolerance ϵ , k indexes `[0, max_iterations)`, $s \in S$ with $|S| = \text{code{n_states}}$, and $V_{k+1}(s)$ is obtained from the action values returned by `compute_action_values`.

For PI, see pgs. 26–28 of [Ref. II](#) and the Policy Iteration page in Ref. I: [Policy Iteration](#). Because PI involves both policy evaluation and policy improvement, we provide a code template with `TODD` markers indicating the required components.

3. Change the `FrozenLake` environment to have stochastic transitions: set the parameter `slippery` to `True` in the `if __name__ == '__main__'` block.

4. By filling in code within the **TODO**-highlighted block in the `compute_action_values` function, modify the computed reward (stored in the `reward` variable) by:
 - (a) adding a positive scalar value to the reward at each step, e.g., $r' = r + 1$.
 - (b) multiplying the reward at each step by a positive scalar value, e.g., $r' = 2r$.
 - (c) adding a penalty for each step, e.g., $r' = r - 0.1$.

Questions

For each algorithm (i.e., value and policy iteration):

1. Compare how the value function and policy change under each reward modification compared to the baseline ($r' = r$). For example, how does the magnitude of V (its norm) vary for each r' modification when contrasted with the baseline case, and does the printed optimal policy change?
2. Building on 1, explain why each reward design does or does not alter the optimal policy.
3. How does each reward modification affect the convergence of each algorithm?
4. How do deterministic vs. stochastic environments (i.e., setting `slippery=True`) influence algorithm performance?

Exercise 2. Tabular Model-Free RL (50 pts.)

In this exercise, you will implement tabular model-free RL algorithms—Q-Learning (off-policy) and SARSA (on-policy)—for a grid-world setting. The provided file `tabular_rl.py` contains the skeletal code for the `FrozenLake` environment where you will implement the algorithms, plot the results, and tune the hyperparameters. The tasks to be completed are as follows:

1. Q-Function Update Logic

Fill in the code for updating the Q-function in the training loop of the `q_learning` function (highlighted with a **TODO** comment). For reference, the update rule from Ref. II (pg. 37) is:

$$Q(S_t, A_t) := Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma \max_{A_{t+1} \in A} (Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)) \right), \quad (2)$$

where Q is the Q-function, represented by the multi-dimensional (`n_states` x `n_actions`) array, `q_table`, α (`alpha`) is the learning rate, γ (`gamma`) is the discount factor, and the maximum operation is computed over the set containing the Q-values corresponding to the set of possible actions that can be taken at the *next state*, S_{t+1} (`next_state`), i.e., `q_table[next_state]`.

2. SARSA Function

- a. Create a new function `sarsa` modeled after `q_learning`, but with the SARSA on-policy update. In `sarsa`, replace the Q-update with the rule from Ref. II (pg. 36):

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha \overbrace{[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]}^{\text{Temporal Difference (TD) Error}}, \quad (3)$$

where ϵ (**epsilon**) is the random-action probability. Use the provided **choose_action** helper to select actions via the ϵ -greedy policy. In particular, the TD error must use the Q-value at the *next* state–action pair $Q(S_{t+1}, A_{t+1})$, where A_{t+1} is selected by calling **choose_action** on **next_state**.

(**Hint:** Duplicate **q_learning**, rename it to **sarsa**, and modify the Q-update using (3).)

- b. Comment out the "sarsa": sarsa entry in the **alg_names** dictionary so that we can specify the **sarsa** implementation from part (a) as our algorithm of choice in 3.

3. Hyperparameter Tuning

For each algorithm, Q-learning and SARSA:

- a. **Run the baseline configuration:**

Plot the reward evolution using:

```
python3 tabular_rl.py --alg <alg_name>
```

where **<alg_name>** is **qlearn** or **sarsa**. A plot is saved automatically (see [Appendix A.1](#)), and summary metrics (see yellow box below) are printed to the terminal. Provide both outputs in your report. (**Note:** You may need to run the script more than once to obtain a non-zero tail success rate and Last-100 avg reward.)

```
METRICS SUMMARY: Q-learning | slippery=False
-----
Full-run avg reward:      0.0100
Full-run variance:       0.0099
Last-100 avg reward:     0.0400
Last-100 variance:       0.0384
Success rate (last 100 eps): 4.0%
Time to first success:    15
```

- b. **Test sensitivity to random seeds:**

Change the evaluation seed for the random number generator, and generate plots using:

```
python3 tabular_rl.py --alg <alg_name> --seed <seed_arg>
```

Run with **<seed_arg>** in {5, 10, 15}. Provide the metrics and plots for each case.

- c. **Tune the hyperparameters:**

Tune each algorithm's hyperparameters (keep the seed fixed at 10; see [Tuning Guide](#)) using:

```
python3 tabular_rl.py --alg <alg_name> --alpha 0.2 --gamma 0.9 \
--epsilon 0.6 --episodes 800 --seed 10
```

Adjust the values until you obtain a satisfactorily-performing configuration, i.e., a run with:

- A high Last-100 avg reward value (≥ 0.90),
- A high value for Success rate (last 100 eps) ($\geq 90\%$), and
- A low Last-100 variance value (≈ 0.00), and
- Stable tail behavior (last ≈ 100 episodes nearly flat; see [Figure 2](#)).

Then record the following in your report:

- i. Plot and metrics for the best Q-learning setting, and
- ii. Plot and metrics for the best SARSA setting,

Note that the hyperparameters should be tuned **separately** for each algorithm; the “optimal” parameter set for Q-learning is *not* necessarily optimal for SARSA. When pasting into VSCode’s integrated terminal, choose Paste (not “Paste as one line”).

4. Effect of Stochastic Transitions

Evaluate your tuned agents under *stochastic* transitions by adding `-slippery True` to your run command. Use the **same hyperparameters** you selected in part 3(c) and a fixed `seed` (set to 10). Ensure the correct `<alg_name>` is specified:

```
python3 tabular_rl.py |
--alg <alg_name> |
--alpha <best_alpha> --gamma <best_gamma> |
--epsilon <best_epsilon> --episodes <best_episodes> |
--seed 10 --slippery True
```

Include in your report both the saved plot and the printed metrics for each algorithm. Stochastic-transition plots will be saved automatically to `plots/stochastic/`.

Based on your results from the above trial runs, answer the following questions:

Questions

1. Using your plots from part 3(a), which algorithm (Q-learning or SARSA) appears more stable on `FrozenLake`? Support your answer using the trends in the curves and printed metrics.
2. Using your results from part 3(b), do different seeds produce similar overall behavior? Briefly explain using the last-100 average reward, variance, or curve shapes.
3. Based on your trials in part 3(c), how did changing the hyperparameters affect performance?
4. Using your results from part 4, how does adding stochasticity affect performance?
5. In the algorithms implemented so far, we explicitly represent the Q-function and value function using tables (multi-dimensional NumPy arrays). What limitations might this approach face in larger or continuous environments?

3 Grading Rubric

The grading rubric is as follows:

Component	Points
Lab Report and Formatting	10
Question 1.	5
Exercise 1.	35
Exercise 2.	50
Total	100

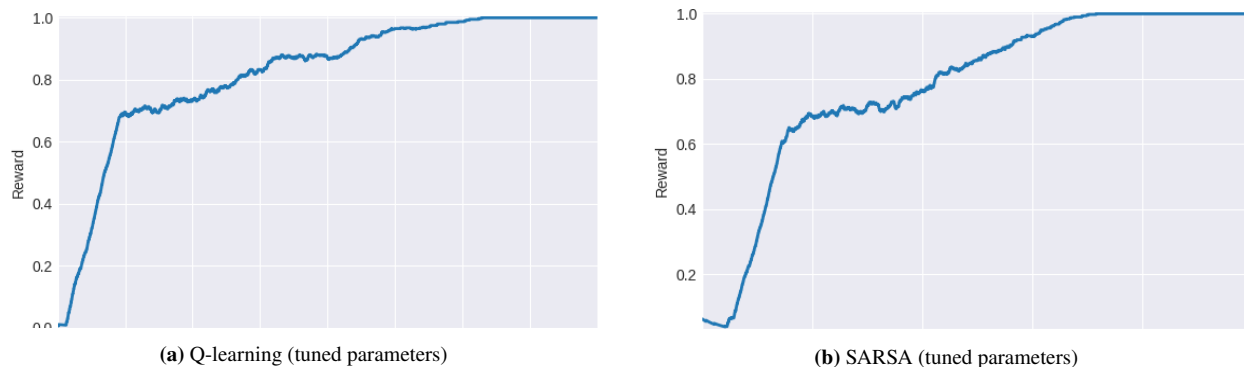


Figure 2: Reward-evolution curves for tuned Q-learning and SARSA. The final two vertical plot grid columns highlight the tail region, showing stable learning.

Appendix

A.1 CLI Program Execution & Plot Generation

- I. By default, running `tabular_rl.py` with **no CLI arguments** uses the `q_learning` algorithm, a `seed` of `0`, and deterministic transitions (i.e., `--slippery False`). It also saves the file `train_qlearn_seed0_alpha0.2_gamma0.9_epsilon0.6_episodes800.png` in `plots/deterministic/`, where the numbers after each keyword parameter is the default value set in the script (i.e., `alpha=0.2`, `gamma=0.9`, `epsilon=0.6`, `episodes=800`). Changing hyperparameters alters the filename.
- II. You may need to click the Refresh Explorer icon (↻) at the top-right corner of VSCode's EXPLORER pane (to the left of the window) to see the saved file. You can then download it from the remote machine by right-clicking on the file and selecting Download or taking a screenshot from the open file and pasting it into your report.

A.2 Hyperparameter Tuning Guide

- I. Learning Rate (`alpha`): larger values (0.7–0.9) learn faster but may be unstable; smaller values (0.1–0.3) are steadier but slow.
- II. Discount factor (`gamma`): high values (0.9–0.99) emphasize long-term reward; low values emphasize short-term gains.
- III. Exploration Rate (`epsilon`): values in the interval $[0.3, 0.5]$ promote exploration; values < 0.3 encourage exploitation, i.e., greedy actions.
- IV. Episode length (`episodes`): more episodes yield smoother curves (use ≥ 3500).
- V. Fixing one parameter and varying the rest is a reasonable strategy, but RL hyperparameters often interact. A change in one can override or mask the effect of another, so effective tuning usually requires adjusting combinations rather than individual values. For more insight, see the following research paper: [What Matters In On-Policy Reinforcement Learning?](#)